

COMP 272 / COMP 400C · DATA STRUCTURES II

Module 3: Trees I

Hierarchies, Binary Trees and BSTs

Original course slides prepared by Prof. Silva, adapting Data Structures & Algorithms in Java (Goodrich, Goldwasser, Tamassia) and Data Structures and Algorithm Analysis in Java (Weiss). Revised and expanded by Dr. Karima Ennaoui for Fall 2026.



What We'll Cover

- | | | |
|----|----------------------------|--|
| 01 | Tree Fundamentals | Terminology, the linked representation, the Tree ADT, and traversals |
| 02 | Binary Trees | The special case of ≤ 2 children — properties and traversal by position |
| 03 | Binary Search Trees | Ordering the tree so search, insert, and delete cost $O(h)$ |

Throughout: every structure is examined for what it costs in the worst case, and why that motivates the next one.

01

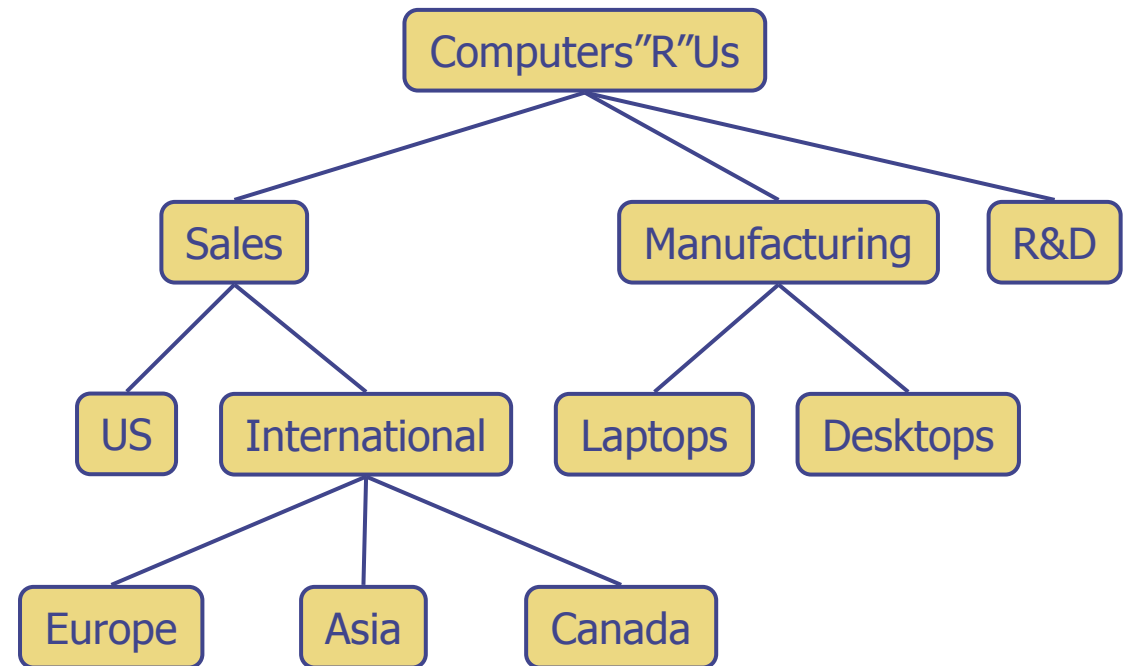
Tree Fundamentals

Hierarchies, vocabulary, the linked representation, the Tree ADT, and traversal order

What Is a Tree?

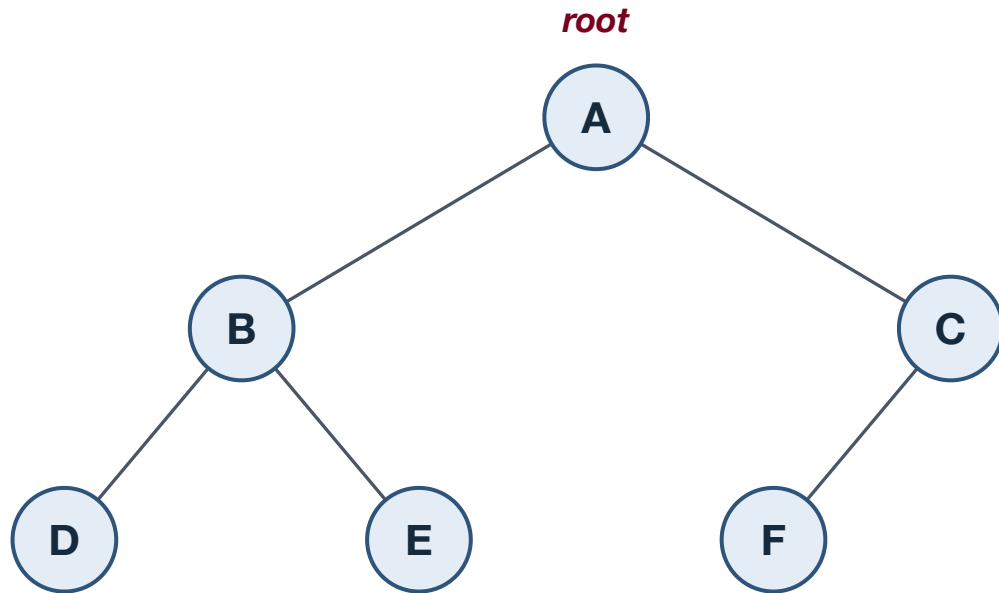
- A tree is an abstract model of a hierarchical structure: nodes connected by a parent–child relationship.
- Exactly one node — the root — has no parent. Every other node has exactly one parent.
- Trees show up constantly in software, not just in data-structures class:
 - File systems (folders contain files and folders)
 - The DOM of every web page you've ever loaded
 - A compiler's parse tree / abstract syntax tree for your Java code
 - Decision trees in machine learning
 - Family trees, org charts, taxonomies, comment threads, JSON/XML documents

Company Organization Tree



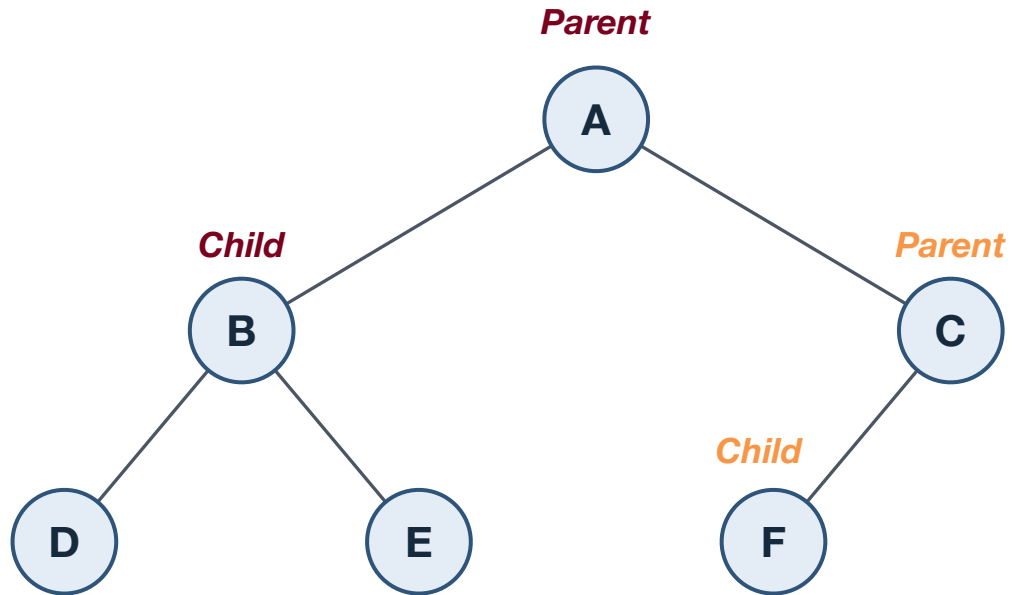


Tree Terminology — Structure



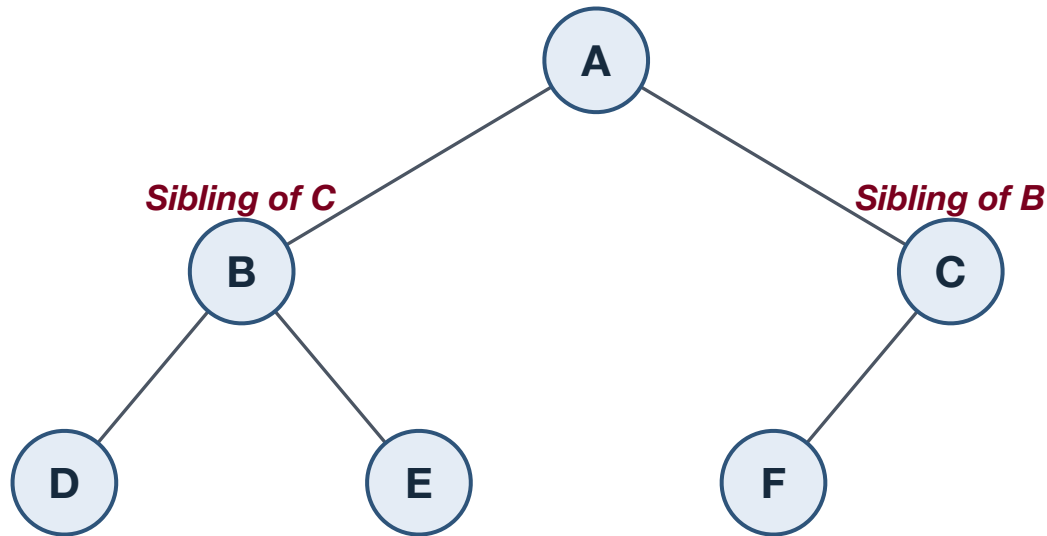
- Root — the node with no parent (A)

Tree Terminology — Structure



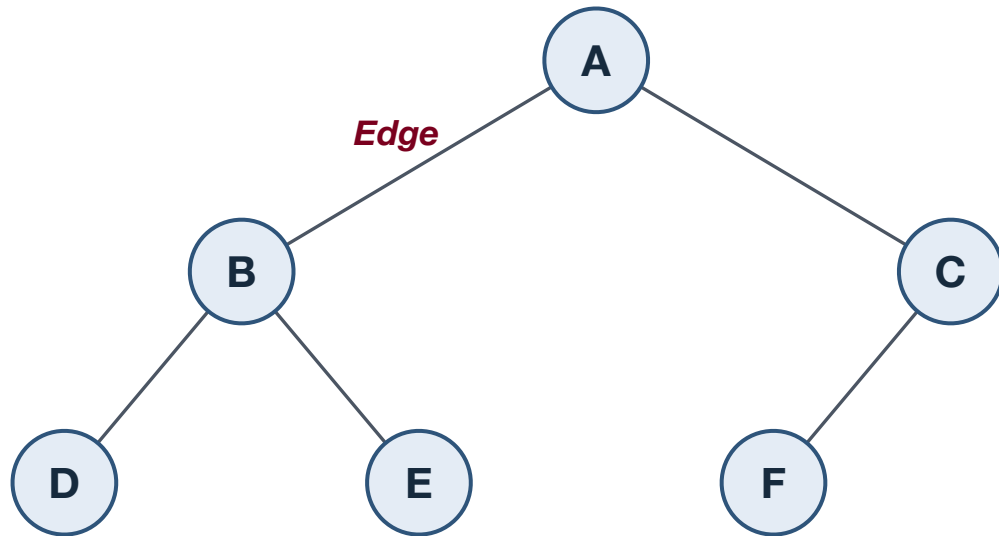
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B

Tree Terminology — Structure



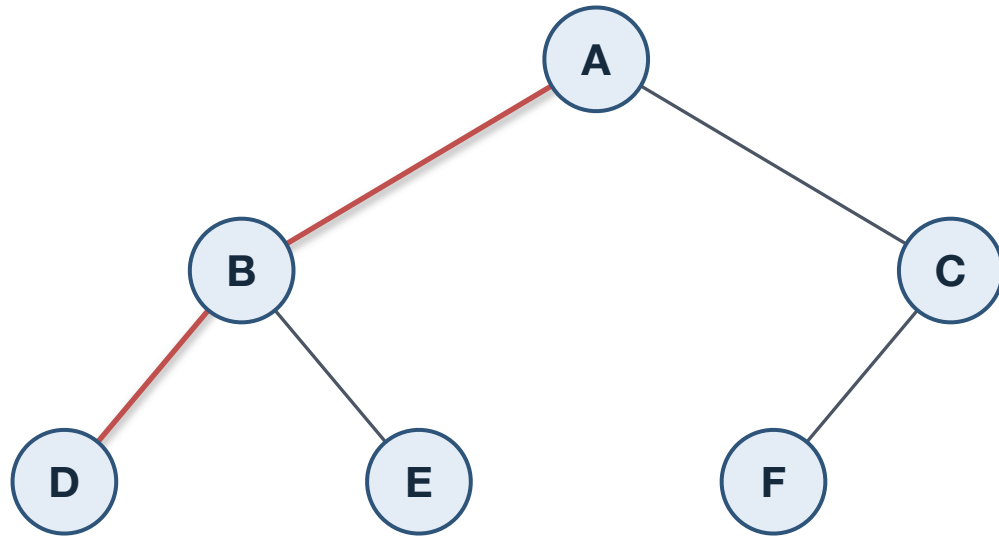
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)

Tree Terminology — Structure



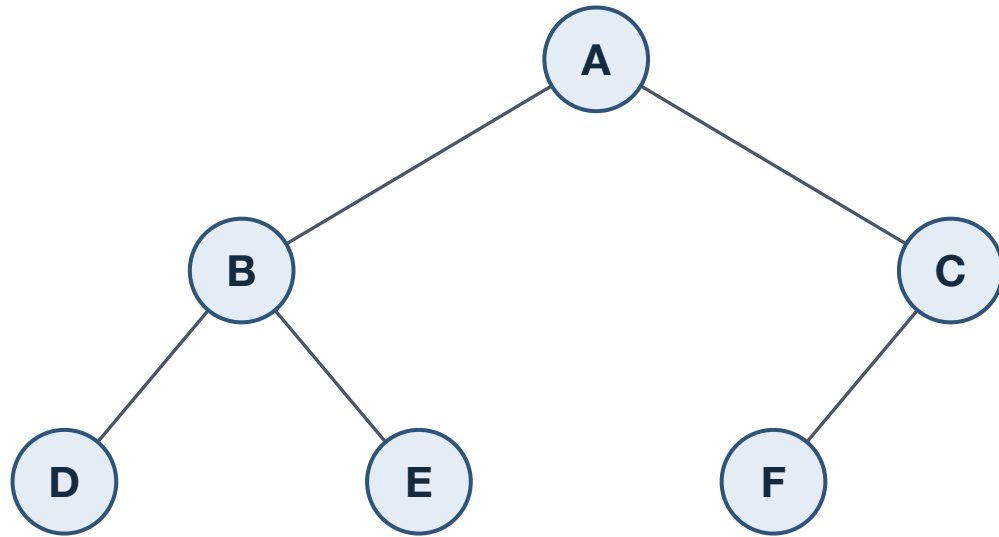
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)
- Edge — the connection between a parent and a child

Tree Terminology — Structure



- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)
- Edge — the connection between a parent and a child
- Path — a sequence of nodes connected by edges, e.g. $A \rightarrow B \rightarrow D$

Tree Terminology — Measuring the Tree

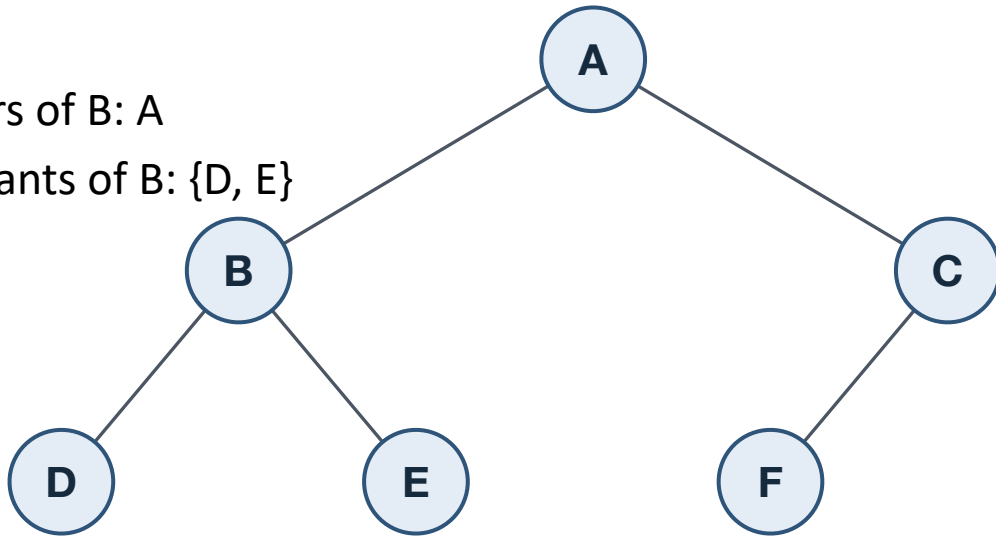


- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)

Tree Terminology — Measuring the Tree

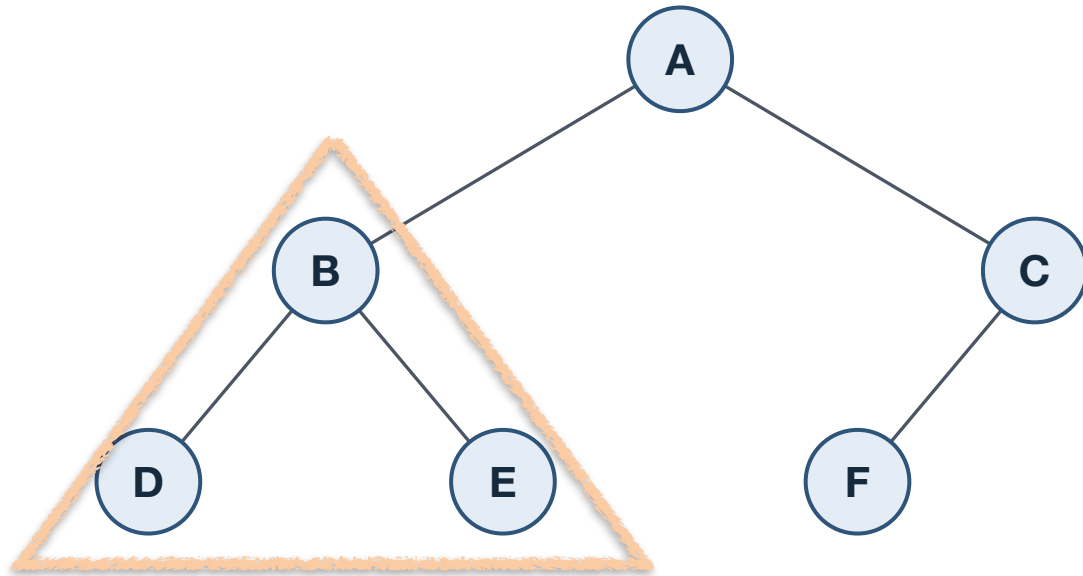
Ancestors of B: A

Descendants of B: {D, E}



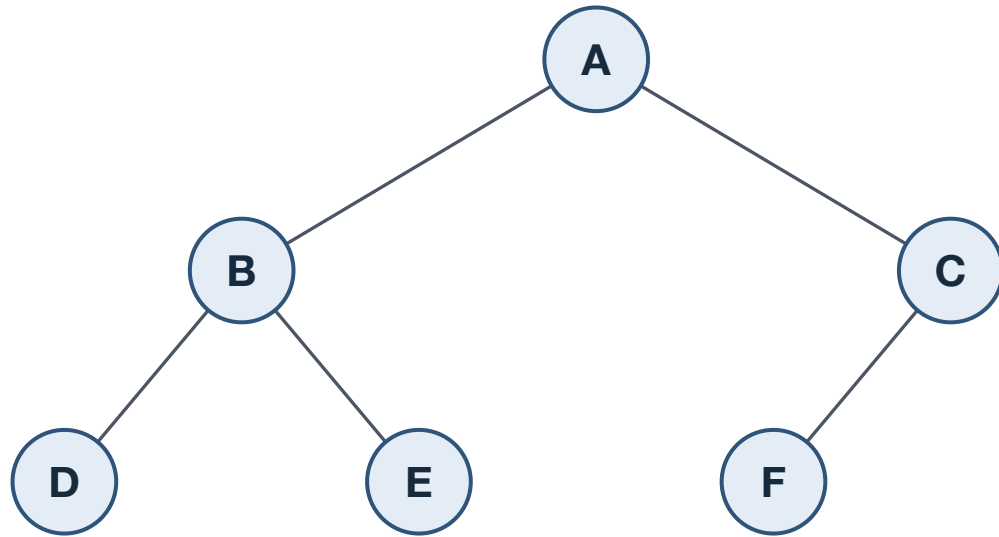
- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...

Tree Terminology — Measuring the Tree



- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...
- Subtree — a node together with all of its descendants (the highlighted subtree rooted at B)

Tree Terminology — Measuring the Tree



Depth of B = 1

Depth of A = 0

Height of the tree = 2

- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...
- Subtree — a node together with all of its descendants (the highlighted subtree rooted at B)
- Depth of a node — number of ancestors
- Height of the tree — max depth of any node



Representing a Tree in Java

TreeNode<T>.java

```
public class TreeNode<T> {  
  
    // Element stored at this node  
    private T element;  
  
    // Reference to the parent  
    // (null if this node is the root)  
    private TreeNode<T> parent;  
  
    // This node's children, in order  
    private List<TreeNode<T>> children;  
  
}
```

- Each node is an object holding three things: its element, a reference to its parent, and a list of references to its children.
- The parent reference is what makes upward navigation (e.g. “what is my depth?”) possible without re-walking the tree from the root.
- A general tree node needs a list because the number of children is unbounded — contrast this with the binary tree node we'll see shortly, which has exactly two named child references.



The Tree ADT

Generic · Accessor

```
int size()
boolean isEmpty()
Iterator iterator()
Iterable nodes()

Node root()
Node parent(p)
Iterable children(p)
```

Query · Update

```
boolean isInternal(p)
boolean isExternal(p)
boolean isRoot(p)

element replace(p, o)
// data structures that implement
// Tree may add more update methods
```

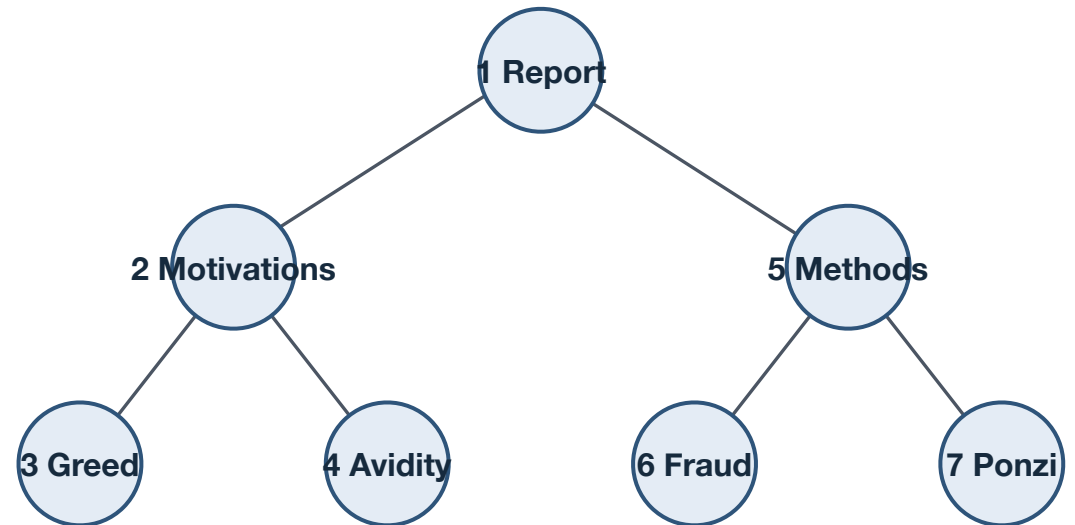
- An Iterator is a cursor: hasNext() checks if there's more, next() advances and returns the next element.
- Iterable just means “can produce an iterator” — it's what makes a for-each loop over the tree possible.

Preorder Traversal

- Visit a node before visiting any of its descendants.
- Application: printing a structured document, where a section must appear before its subsections.

```
Algorithm preOrder(v)
  visit(v)
  for each child w of v
    preOrder(w)
```

Visit order:

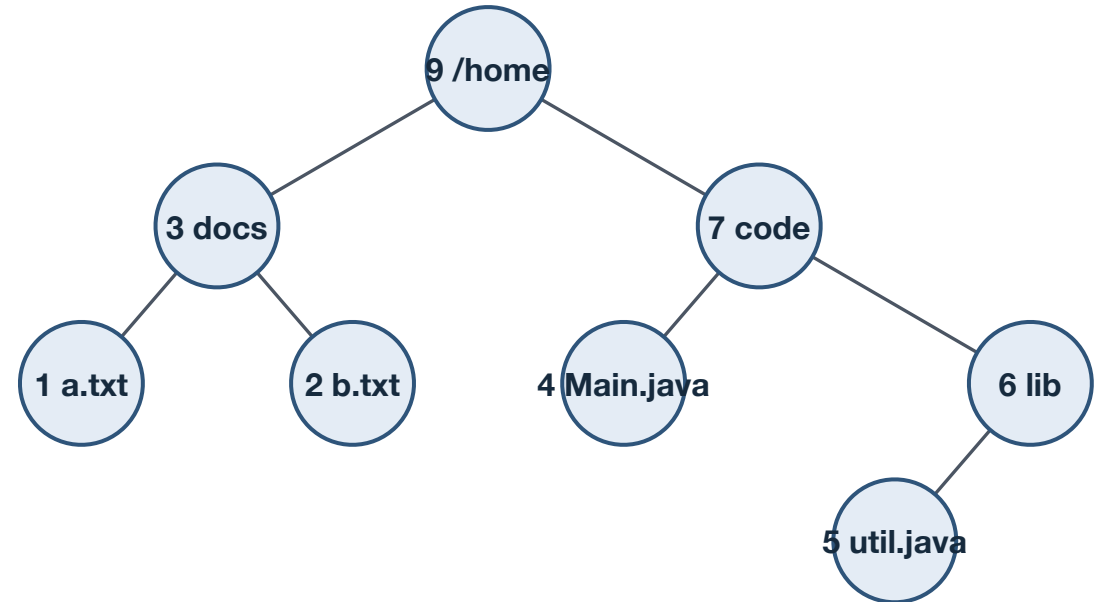


Postorder Traversal

- Visit a node after visiting all of its descendants.
- Application: computing the total disk space used by a directory — you need every subdirectory's total before you can report the parent's.

```
Algorithm postOrder(v)
  for each child w of v
    postOrder(w)
  visit(v)
```

Visit order (folder sizes computed bottom-up):





Which Traversal Do I Need?

Preorder*Visit node, then children*

Copying/serializing a tree; printing outline-style structure

Postorder*Visit children, then node*

Deleting a tree safely; computing a value that depends on all descendants (folder size, subtree height)

Inorder — binary trees only*Visit left subtree, node, right subtree*

Reading a binary search tree's keys out in sorted order (Section 03)

All three visit every node exactly once, so all three run in $\Theta(n)$ time regardless of the tree's shape.

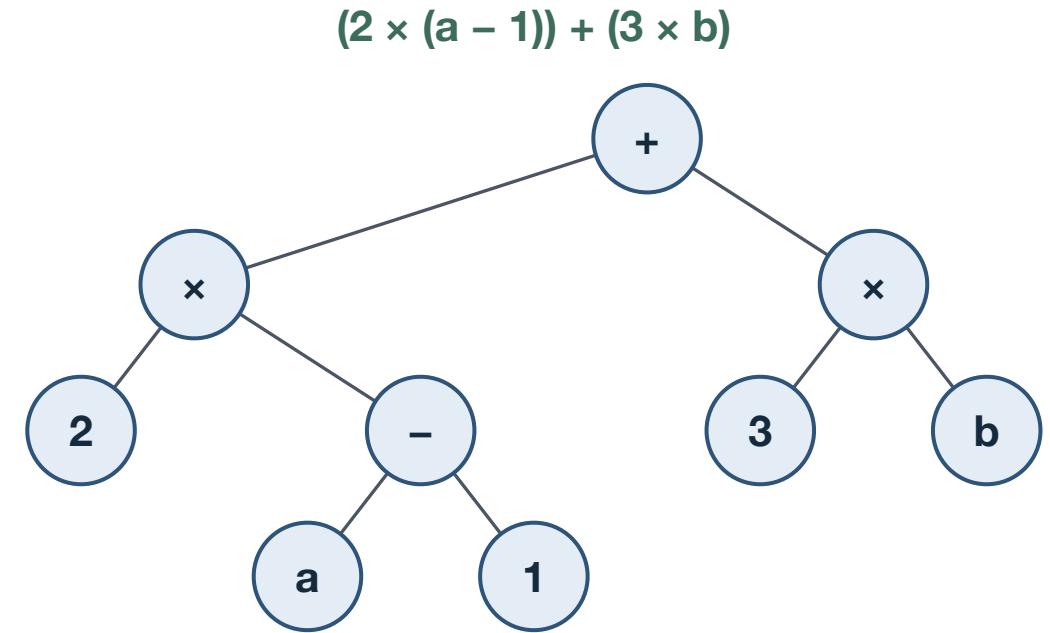
02

Binary Trees

The special case where every node has at most two, ordered children

Binary Trees

- A binary tree is a tree in which every internal node has at most two children.
- The two children are an ordered pair — we call them the left child and the right child, and a node with only one child still distinguishes which side it's on.
- Proper (a.k.a. full) binary tree: every internal node has exactly two children.
- Applications:
 - Arithmetic expression trees (below)
 - Decision processes (yes/no questions)
 - Searching — the entire rest of this module





Properties of Proper Binary Trees

Notation

- n — number of nodes
- e — number of external nodes (leaves)
- i — number of internal nodes
- h — height

Read the last line as the payoff: with n nodes, height can be as low as about $\log_2(n)$ — but nothing here forces it to be that low. A tree can just as easily be a straight chain with $h = n-1$. That gap is exactly what Sections 04–06 close.

Because every internal node has exactly 2 children:

- $e = i + 1$
- $n = 2e - 1$
- $h \leq i$
- $h \leq (n - 1) / 2$
- $e \leq 2^h$
- $h \geq \log_2 e$
- $h \geq \log_2(n + 1) - 1$



The BinaryTree ADT

BinaryTree extends Tree — it inherits every Tree method, plus position-aware accessors:

```
Node left(p)
Node right(p)
boolean hasLeft(p)
boolean hasRight(p)
```

This is the key upgrade over the general Tree ADT: children now have identity (left vs. right), not just membership in an unordered list. Every algorithm for the rest of this module leans on that distinction.

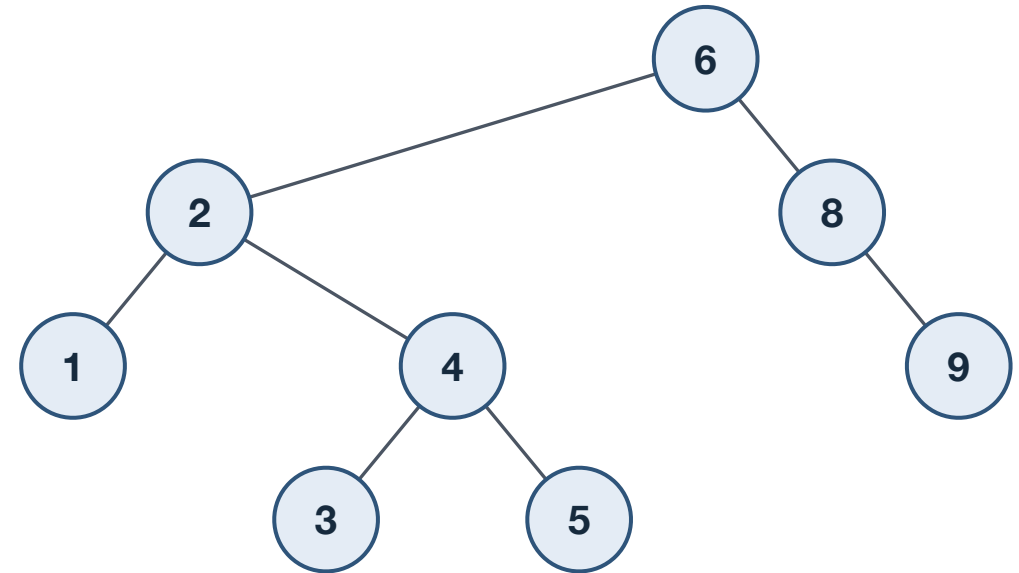


Inorder Traversal

- Visit the left subtree, then the node itself, then the right subtree.

```
Algorithm inOrder(v)
  if hasLeft(v)
    inOrder(left(v))
  visit(v)
  if hasRight(v)
    inOrder(right(v))
```

Visits 1, 2, 3, 4, 5, 6, 8, 9 — in order:





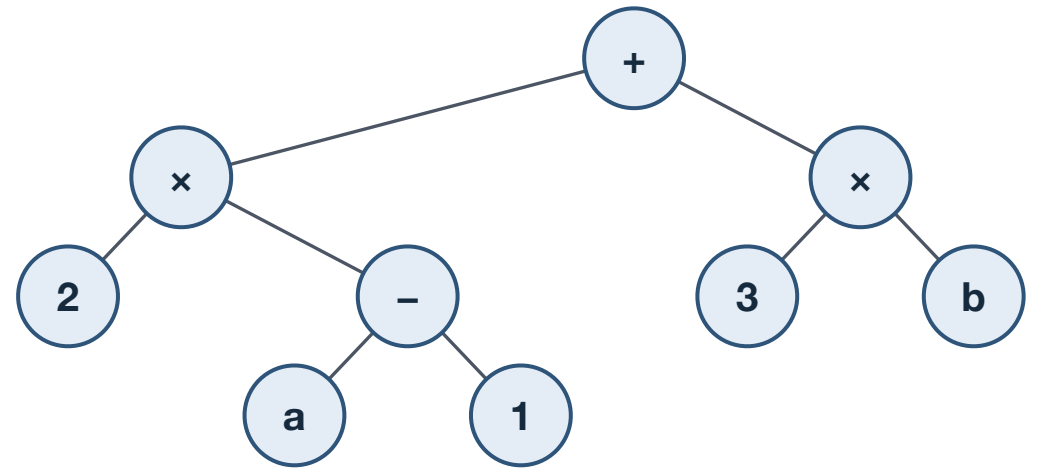
Application: Printing Expressions

- A specialization of inorder traversal: print "(" before the left subtree and ")" after the right subtree, and print the node's value when visiting it.

```
Algorithm printExpr(v)
  if hasLeft(v) print("(")
  printExpr(left(v))
  print(v.element())
  if hasRight(v)
    printExpr(right(v))
  print(")")
```

Output:

((2 × (a − 1)) + (3 × b))

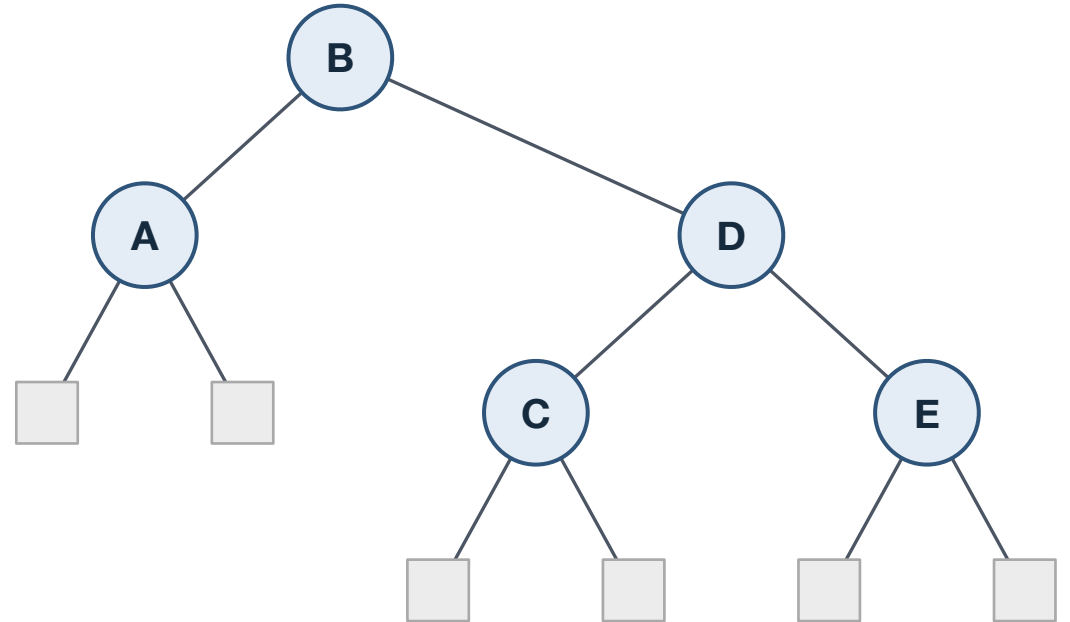




The Linked Representation, Extended

BTNode<T>.java

```
public class BTNode<T> {  
  
    private T element;  
    private BTNode<T> parent;  
  
    // Named left/right instead of a  
    // list of children  
    private BTNode<T> left;  
    private BTNode<T> right;  
  
}
```

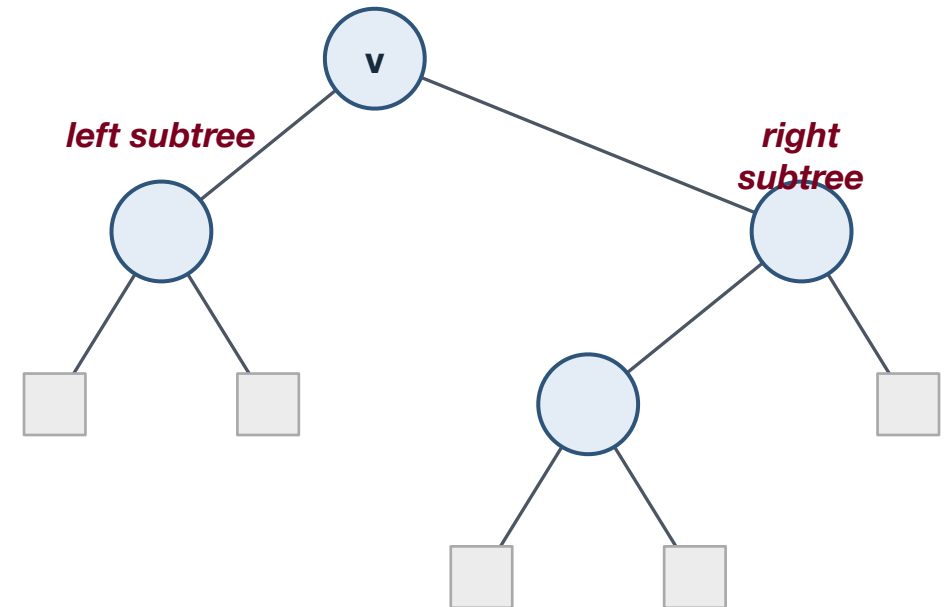


Same idea as the general tree node, but left/right replace the children list — $O(1)$ access to either child instead of a list scan.



Example: Computing Height

```
Algorithm height(T, v)
  if v is an external node
    return 0
  else
    x = y = 0
    if hasLeft(v)
      x = height(T, left(v))
    if hasRight(v)
      y = height(T, right(v))
    return max(x, y) + 1
```



Every node is visited once and does $O(1)$ work outside its recursive calls — so this costs $O(n)$, even though the height it computes can be $O(\log n)$.

03

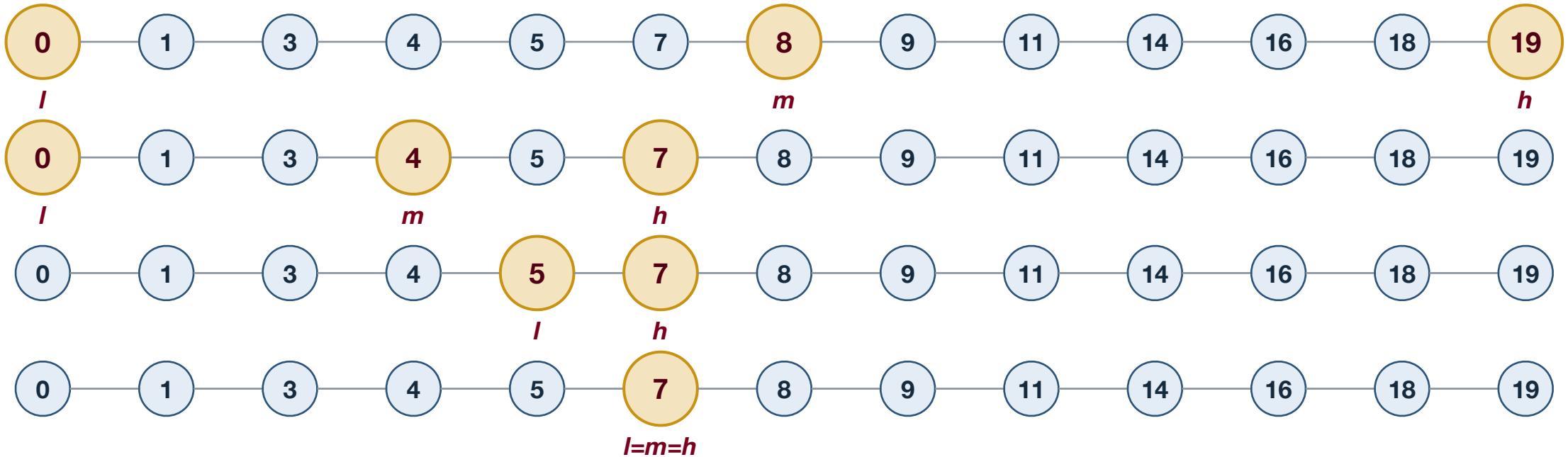
Binary Search Trees

Ordering the tree so that search, insertion, and deletion all cost $O(h)$



Warm-Up: Binary Search

- On a sorted array, binary search finds a key the way you'd play a high-low guessing game: check the middle, then discard half the remaining candidates.
- Each step halves the candidates, so it terminates in $O(\log n)$ steps. Example: find(7).

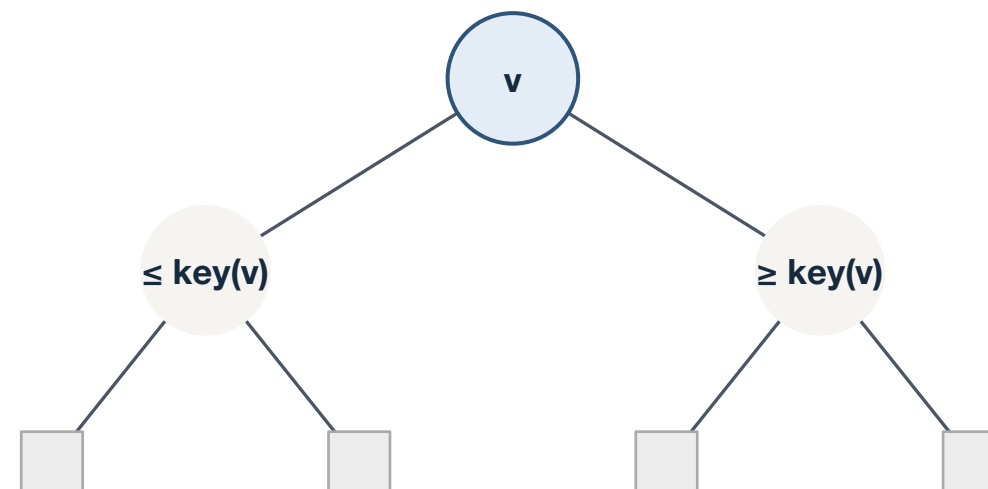


A binary search tree gives the same halving behavior to a linked structure — no array shifting required on insert/delete.



The BST Ordering Property

- A binary search tree is a proper binary tree where every internal node stores a key, and for every node v :
 - every key in v 's left subtree is $\leq \text{key}(v)$
 - every key in v 's right subtree is $\geq \text{key}(v)$
- External (leaf) nodes store no key — they're placeholders marking “no child here,” shown below as gray squares.
- Consequence: an inorder traversal visits the keys in increasing order — for free, with no extra bookkeeping.

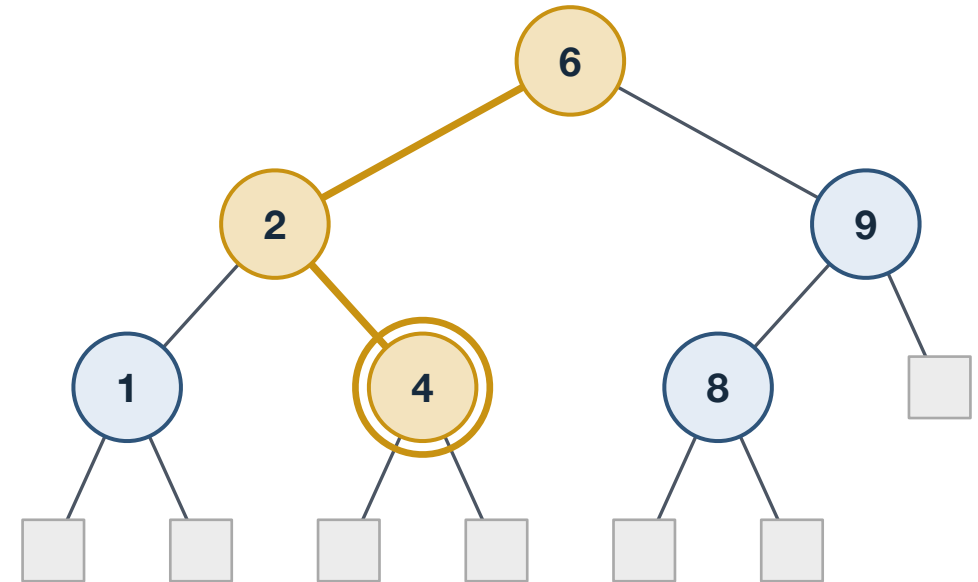


BINARY SEARCH TREES

Search — Algorithm

```
Algorithm treeSearch(k, v)
  if T.isExternal(v)
    return v          // key not found
  if k < key(v)
    return treeSearch(k, left(v))
  else if k = key(v)
    return v          // found
  else
    return treeSearch(k, right(v))
```

Example: get(4)

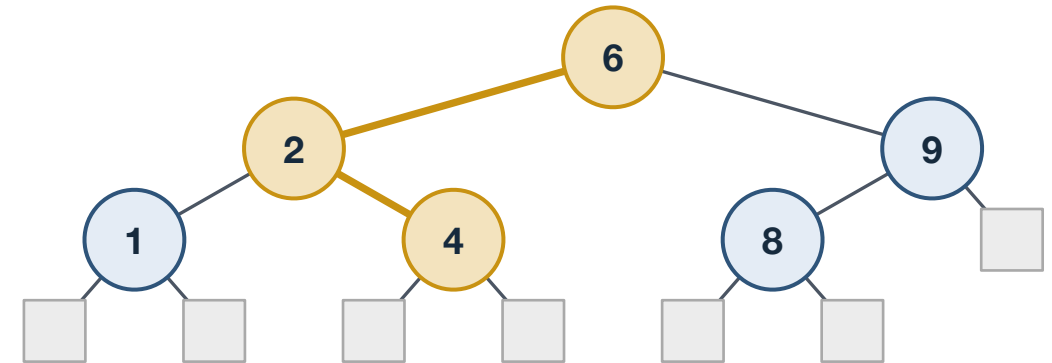


- Trace a single downward path from the root; at each node, the comparison tells you which subtree could possibly contain k .
- If the path runs off the tree onto an external node, k is not present — and that external node is exactly where k would be inserted.

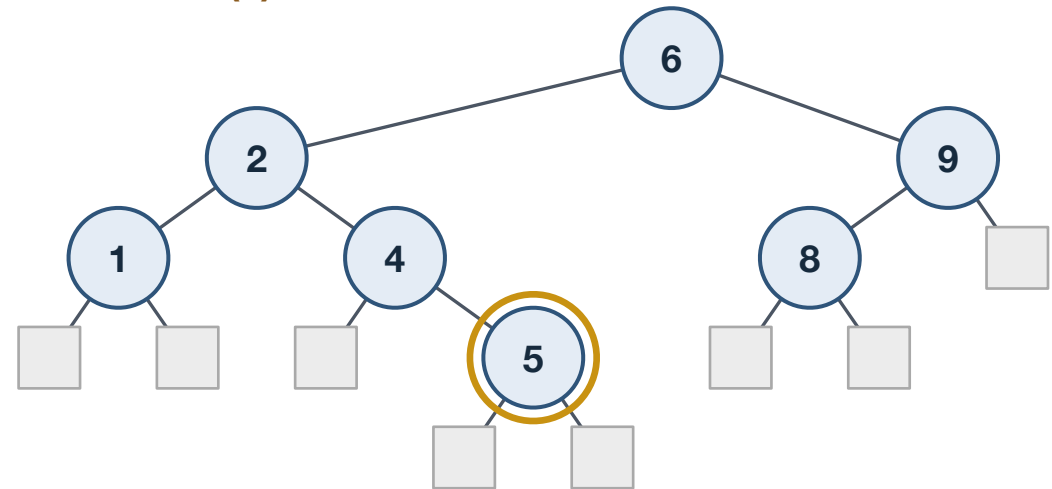


Insertion — Algorithm

- To insert key k ($\text{put}(k, o)$):
 - Run $\text{treeSearch}(k, \text{root})$ — assuming k isn't already present, this ends at an external node w .
 - Insert k at w and expand w into an internal node with two new external children.
- That's it — insertion always happens at a former leaf position, which is why it costs $O(h)$ just like search.



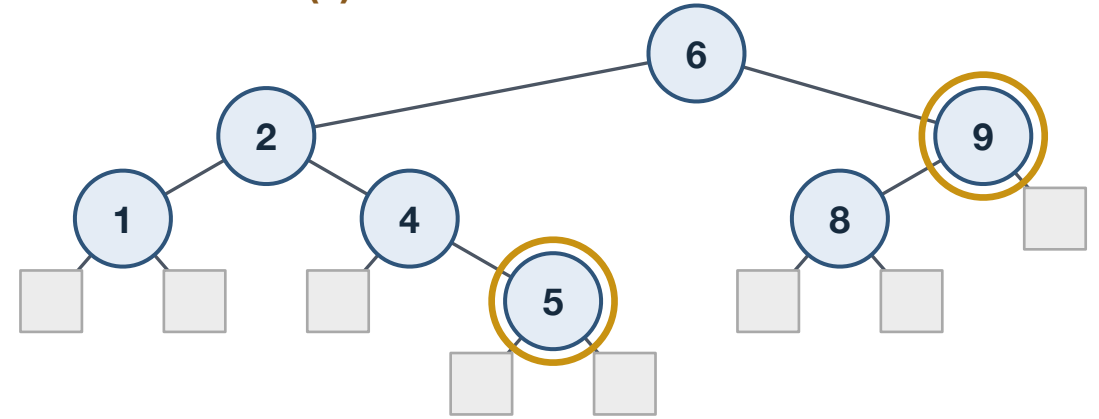
After insert(5):



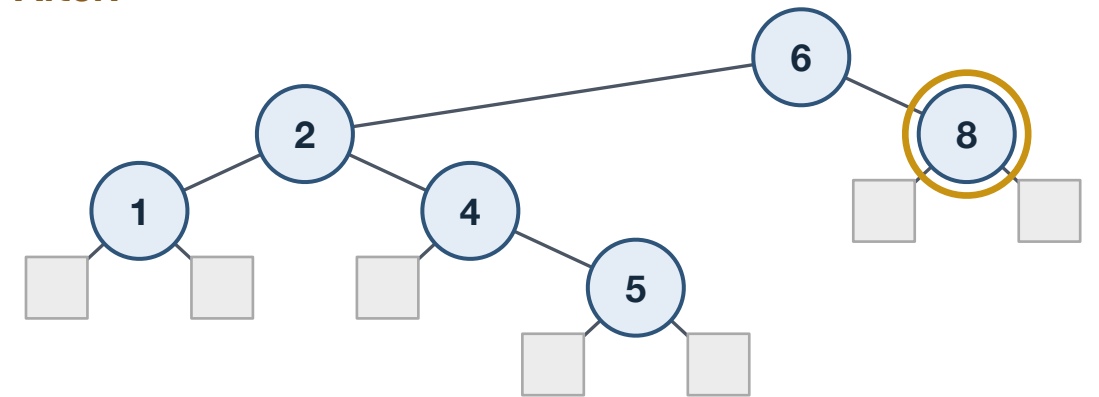
Deletion — Easy Case: a Leaf Child

- To remove key k : search for k , landing at the node v that stores it.
- If at least one of v 's children is external (a leaf placeholder) — call it w — `removeExternal( $w$ )` deletes both w and v , and v 's other child slides up to take v 's place.
- Example: `remove(9)`. Node 9's right child is external, so 9 is removed outright and its left child, 8, is promoted.

Before: `remove(9)`



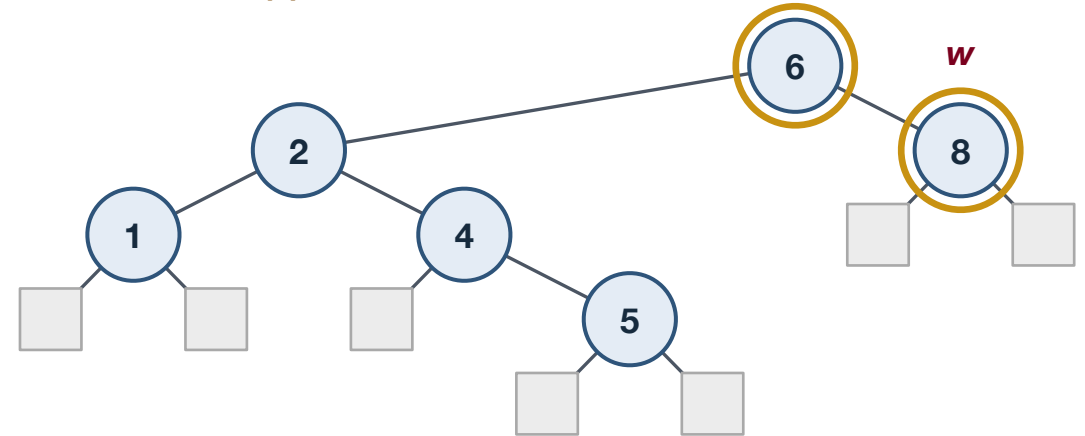
After:



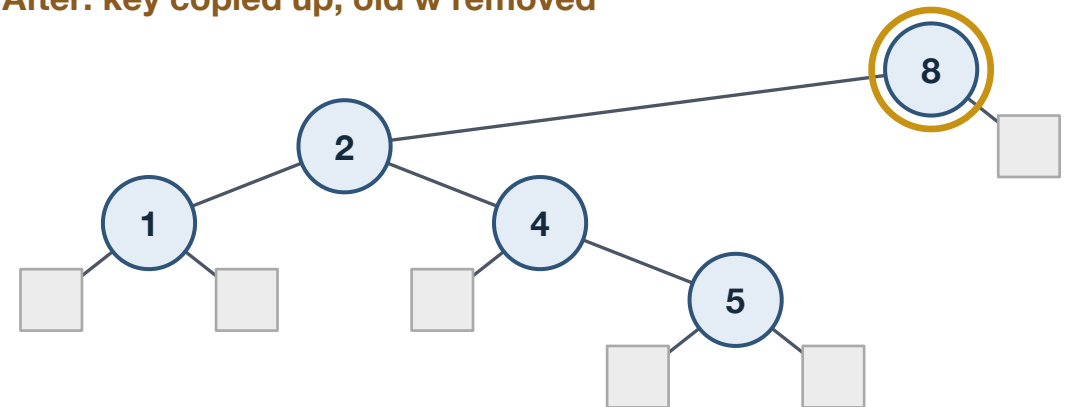
Deletion — Hard Case: Two Internal Children

- If both of v 's children are internal, we can't just promote one — we need a replacement key that preserves the ordering property.
- Find w , the node immediately after v in an inorder traversal (its inorder successor) — concretely, go right once, then left as far as possible. w is guaranteed to have an external left child.
- Copy $\text{key}(w)$ into v , then delete w using the easy case above.
- Example: $\text{remove}(6)$ from the tree at right. The inorder successor of 6 is 8; copy 8 into the root, then remove the old node 8.

Before: $\text{remove}(6)$, $v = \text{root}$, $w = \text{successor}$



After: key copied up, old w removed

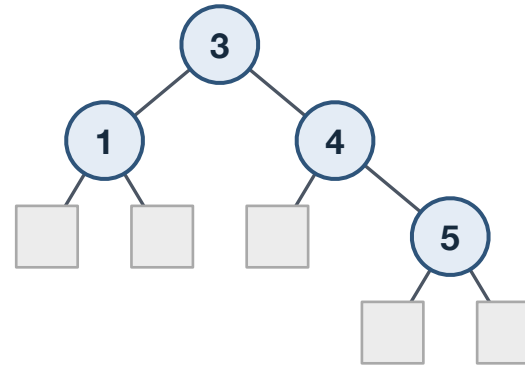


Performance — and a Problem

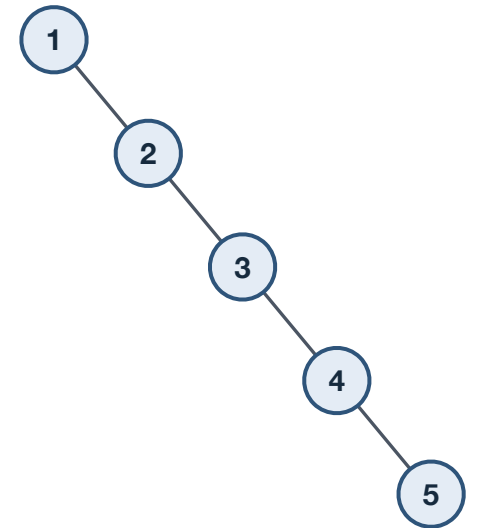
- For a BST with n nodes and height h :
 - space used: $O(n)$
 - get, put, remove: all $O(h)$, since each does at most one root-to-leaf pass
- The catch: h depends entirely on insertion order.

Insert keys already sorted — 1, 2, 3, 4, 5 — and every node has only a right child. The “tree” degenerates into a linked list: $h = n - 1$, and every operation becomes $O(n)$.

Balanced ($n=5, h=2$)



Sorted insert ($n=5, h=4$)



This is exactly the gap AVL (Sec. 04) and Red-Black trees (Sec. 05) close: both guarantee $h = O(\log n)$ regardless of insertion order.